🏗️ Builder Design Pattern

Complete Guide with Java 17 Implementation

📖 What is Builder Pattern?

> **Definition:** A creational design pattern that separates the construction of a complex object from its representation, allowing the same construction process to create different representations.

Think of it like ordering a custom pizza — you start with a base and add toppings one by one, and at the end, you get your complete pizza. The Builder pattern works the same way for creating objects.

Pattern Classification

Attribute	Value
Pattern Type	Creational Pattern
Purpose	Object Construction
Complexity	Medium
Popularity	⭐⭐⭐⭐⭐ Very High

❓ Why Builder Pattern is Needed?

Problem 1: Telescoping Constructor Anti-Pattern

```java
public class User {

    // Constructor explosion - Hard to maintain!

    public User(String firstName) { }

    public User(String firstName, String lastName) { }

    public User(String firstName, String lastName, int age) { }

    public User(String firstName, String lastName, int age, String e

    public User(String firstName, String lastName, int age, String e
}

// Usage - Which parameter is which? 🤔
User user = new User("John", null, 25, null, "12345");
```

Problems with this approach:

- Too many constructor combinations (2^n for n optional fields)
- Hard to read — unclear which parameter is which
- Forced to pass `null` for optional fields
- Easy to swap parameters of same type (silent bugs)

Problem 2: JavaBean Pattern Issues

```java
// JavaBean approach - Mutable and inconsistent state

User user = new User();

user.setFirstName("John");

user.setLastName("Doe");

// Object is in INCONSISTENT state here! ⚠️
```

```
// What if another thread reads it now?

user.setAge(25);

user.setEmail("john@example.com");
```

Problems with JavaBean:

- Object is mutable — can be changed after creation
- Inconsistent state during construction
- Not thread-safe
- No way to enforce required fields

✅ Builder Pattern Solution

JAVA

```
// Clean, readable, immutable!

User user = new User.Builder("John", "Doe")  // Required fields
    .age(25)                                 // Optional
    .email("john@example.com")               // Optional
    .phone("123-456-7890")                   // Optional
    .build();                                 // Create immutable
```

Benefits:

- Crystal clear which value goes where
- Only set what you need
- Immutable object after creation
- Validation before object creation

⏰ When to Use Builder Pattern?

✅ Use Builder When

- Object has 4+ constructor parameters
- Many optional parameters exist
- Immutability is required
- Complex validation needed
- Object creation is multi-step process
- Same construction for different representations

❌ Don't Use When

- Simple objects (2-3 fields)
- All fields are required
- Mutable objects are acceptable
- Performance is critical (slight overhead)
- Simple POJO with setters suffices

Real-World Use Cases

Use Case	Example
HTTP Requests	HttpRequest.newBuilder().uri().header().build()
Database Queries	QueryBuilder for dynamic SQL construction
UI Components	AlertDialog.Builder in Android
Configuration Objects	ServerConfig, DatabaseConfig
Test Data	TestUserBuilder for unit tests

How Builder Pattern Works?

Pattern Flow

Components of Builder Pattern

Component	Role	Description
Product	User	The complex object being built
Builder	User.Builder	Constructs parts of the product
Director	Client Code	Controls the building process

Complete Code Implementation

JAVA

```java
public class User {

    // ═══════════════════════════════════════════════════════════
    // SECTION 1: FIELDS (All final for immutability)
    // ═══════════════════════════════════════════════════════════

    // Required fields
    private final String firstName;

    private final String lastName;

    // Optional fields
    private final int age;

    private final String email;

    private final String phone;

    private final String address;


    // ═══════════════════════════════════════════════════════════
    // SECTION 2: PRIVATE CONSTRUCTOR
    // ═══════════════════════════════════════════════════════════

    private User(Builder builder) {

        this.firstName = builder.firstName;

        this.lastName = builder.lastName;

        this.age = builder.age;

        this.email = builder.email;

        this.phone = builder.phone;
```

```
        this.address = builder.address;
    }


    // ═══════════════════════════════════════════════════════
    // SECTION 3: GETTERS ONLY (No setters = Immutable)
    // ═══════════════════════════════════════════════════════

    public String getFirstName() {
        return firstName;
    }

    public String getLastName() {
        return lastName;
    }

    public int getAge() {
        return age;
    }

    public String getEmail() {
        return email;
    }

    public String getPhone() {
        return phone;
    }

    public String getAddress() {
        return address;
    }

    @Override
    public String toString() {

        return "User{firstName='%s', lastName='%s', age=%d, email='%
            .formatted(firstName, lastName, age, email);
    }
```

```java
    // ═══════════════════════════════════════════════════════════
    // SECTION 4: STATIC BUILDER CLASS
    // ═══════════════════════════════════════════════════════════

    public static class Builder {

        // Required fields (no default value)
        private final String firstName;

        private final String lastName;

        // Optional fields (with default values)
        private int age = 0;

        private String email = "";

        private String phone = "";

        private String address = "";


        // ─────────────────────────────────────────────────────────
        // Builder Constructor (Required fields only)
        // ─────────────────────────────────────────────────────────

        public Builder(String firstName, String lastName) {

            this.firstName = firstName;

            this.lastName = lastName;
        }


        // ─────────────────────────────────────────────────────────
        // Setter Methods (Return Builder for chaining)
        // ─────────────────────────────────────────────────────────

        public Builder age(int age) {

            this.age = age;
```

```
            return this;   // Key for method chaining!
        }

        public Builder email(String email) {

            this.email = email;

            return this;
        }

        public Builder phone(String phone) {

            this.phone = phone;

            return this;
        }

        public Builder address(String address) {

            this.address = address;

            return this;
        }


        // ------------------------------------------------------------
        // Build Method (Creates final immutable object)
        // ------------------------------------------------------------

        public User build() {

            validateUser();   // Validate before creating

            return new User(this);   // Pass builder to User constru
        }


        // ------------------------------------------------------------
        // Validation Logic
```

```
        // ————————————————————————————————————————————————————————————

        private void validateUser() {

            if (firstName == null || firstName.isBlank()) {

                throw new IllegalStateException("First name is requi
            }

            if (lastName == null || lastName.isBlank()) {

                throw new IllegalStateException("Last name is requir
            }

            if (age < 0) {

                throw new IllegalStateException("Age cannot be negat
            }

            if (!email.isEmpty() && !email.contains("@")) {

                throw new IllegalStateException("Invalid email forma
            }
        }
    }
}
```

📝 Line-by-Line Code Explanation

Section 1: Fields Declaration

Fields private final

`private final String firstName;`

Why `private`? Encapsulation — fields cannot be accessed directly from outside the class.

Why `final`? Immutability — once set in constructor, these values can NEVER change. This makes the object thread-safe and predictable.

Section 2: Private Constructor

Constructor private

`private User(Builder builder)`

Why `private`? Prevents direct instantiation with `new User()`. Forces everyone to use the Builder.

Why accept `Builder`? The Builder collects all field values, then passes itself to this constructor. User extracts values from Builder.

Assignment

`this.firstName = builder.firstName;`

What happens? Values are copied from Builder to User fields. After this, the User object is complete and immutable.

Why copy instead of reference? Ensures User doesn't depend on Builder after construction. Builder can be garbage collected.

Section 3: Getters Only

Getters public

`public String getFirstName() { return firstName; }`

Why only getters? No setters means no way to modify the object after creation = **Immutability**.

Thread safety: Multiple threads can safely read this object simultaneously without synchronization.

Section 4: Static Builder Class

Builder Class | public static

```
public static class Builder
```

Why `static`? Allows creation without an existing User instance: `new User.Builder()`.

Without static, you'd need: `new User().new Builder()` which is impossible since User constructor is private!

Why nested inside User? Logical grouping — Builder is specifically for User. Also gives Builder access to User's private constructor.

Required Fields | final

```
private final String firstName;
```

Why `final` in Builder? Required fields must be set in Builder's constructor and cannot be changed later.

Optional Fields | optional

```
private int age = 0;
```

Why default values? Optional fields have sensible defaults. If user doesn't call `.age()`, the default (0) is used.

Why NOT final? These can be modified by setter methods before `build()` is called.

Builder Constructor

```
public Builder(String firstName, String lastName)
```

Why required params in constructor? Enforces that required fields MUST be provided. You cannot create a Builder without them.

Compile-time safety: `new User.Builder()` won't compile — forces `new User.Builder("John", "Doe")`.

Setter Methods

`public Builder age(int age) { this.age = age; return this; }`

Why return `this`? Enables **method chaining** (fluent API):

`.age(25).email("x@y.com").phone("123")`

Each method returns the same Builder instance, allowing the next method to be called on it.

Build Method

`public User build() { validateUser(); return new User(this); }`

Why `validateUser()` here? Single point of validation. All rules checked BEFORE object creation. Invalid objects can never exist.

Why `return new User(this)`? Builder passes itself to User's private constructor. User extracts all values and creates the final immutable object.

Validation

`private void validateUser()`

Centralized validation: All business rules in one place. Easy to maintain and test.

Fail-fast: Throws exception immediately if validation fails. No partially constructed objects.

🚀 Usage Examples

JAVA

```java
public class BuilderDemo {

    public static void main(String[] args) {

        // Example 1: Only required fields
        User user1 = new User.Builder("John", "Doe")
            .build();


        // Example 2: Required + some optional
        User user2 = new User.Builder("Jane", "Smith")
            .age(25)
            .email("jane@example.com")
            .build();


        // Example 3: All fields
        User user3 = new User.Builder("Bob", "Wilson")
            .age(30)
            .email("bob@example.com")
            .phone("123-456-7890")
            .address("123 Main Street")
            .build();


        // Example 4: Order doesn't matter!
        User user4 = new User.Builder("Alice", "Brown")
            .phone("999-888-7777")   // Can set in any order
            .age(28)
            .address("456 Oak Ave")
            .build();


        // Example 5: Validation in action
        try {

            User invalid = new User.Builder("", "Test")   // Empty f
                .build();
```

```
        } catch (IllegalStateException e) {

            System.out.println("Validation failed: " + e.getMessage(
            // Output: Validation failed: First name is required
        }
    }
}
```

Output:

```
User{firstName='John', lastName='Doe', age=0, email=''}
User{firstName='Jane', lastName='Smith', age=25, email='jane@example.com'}
User{firstName='Bob', lastName='Wilson', age=30, email='bob@example.com'}
Validation failed: First name is required
```

Real-World Java Examples

JAVA

```
// 1. StringBuilder (Java Core)

String result = new StringBuilder()
    .append("Hello")
    .append(" ")
    .append("World")
    .toString();


// 2. HttpRequest (Java 11+)

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.example.com/users"))
    .header("Content-Type", "application/json")
    .header("Authorization", "Bearer token123")
    .timeout(Duration.ofSeconds(10))
    .GET()
    .build();


// 3. Stream API (Functional Builder Pattern)
```

```java
List<String> filtered = names.stream()
    .filter(name -> name.startsWith("J"))
    .map(String::toUpperCase)
    .sorted()
    .collect(Collectors.toList());


// 4. ProcessBuilder (Java Core)

Process process = new ProcessBuilder()
    .command("ls", "-la")
    .directory(new File("/home"))
    .redirectErrorStream(true)
    .start();


// 5. Lombok @Builder (Annotation-based)

@Builder
@Getter
public class Employee {

    private String name;

    private String department;

    @Builder.Default
    private int salary = 50000;
}


// Lombok generates builder automatically!

Employee emp = Employee.builder()
    .name("John")
    .department("Engineering")
    .build();
```

📋 Quick Reference Summary

Aspect	Details
Pattern Type	Creational
Main Purpose	Construct complex objects step-by-step
Key Benefit	Readable, maintainable object creation
Immutability	Yes — final fields, no setters
Thread Safety	Yes — immutable objects are inherently thread-safe
Validation	Centralized in build() method
Method Chaining	Enabled by returning `this`
Required Fields	Passed to Builder constructor
Optional Fields	Set via fluent setter methods

💡 **Remember:** Use Builder when you have 4+ parameters, many optional fields, or need immutable objects with validation. For simple objects, regular constructors are fine!